manipulate objects in DOM tree or enable other functionalities. Since these three parts require different programming languages, generating input can be challenging. Moreover, the three parts often have data dependencies. For example, HTML code defines an element, and JavaScript code can then access the element in the DOM tree by indexing it using `document.children[i]`. Once the element is accessed, JavaScript can manipulate it using browser APIs [mozilla 2021].

Due to the complex structured input format, browser fuzzers [Dinh et al. 2021; Google 2017; Mozilla 2015; Xu et al. 2020; Zhou et al. 2022] heavily rely on custom grammars, typically in the format of context-free grammars (CFGs), to generate test cases. A **CFG** is defined as a tuple $G = (N, T, P, S)$ of (1) a set of nonterminals N; (2) a set of terminals T disjoint from N; (3) a set of production rules P, which is a finite relation in $N \times (N \cup T)^k$ for some $k > 0$, in the form of $\alpha \rightarrow \beta_1\beta_2 \ldots \beta_k$, mapping a nonterminal $n \in N$ to an expansion alternative; (4) a designed start symbol $S \in N$. In general, fuzzers leverage production rules P to perform expansions starting with S, and generate a derivation tree for input generation in the end. We use the classical definition of derivation trees as given in literature [Aho et al. 2007] and in line with other grammar-based fuzzing research [Gopinath et al. 2021; Havrikov and Zeller 2019]. In short, a **derivation tree** for $G = (N, T, P, S)$ is defined as a rooted ordered tree such that (1) the root is S; (2) each node is a string that contain multiple symbols $[s_1, s_2, \ldots, s_k]$ where $s_i \in N \cup T$ and $k \geq 0$ (3) each edge is a production rule $p \in P$; (4) all leaves do not contain any nonterminals.

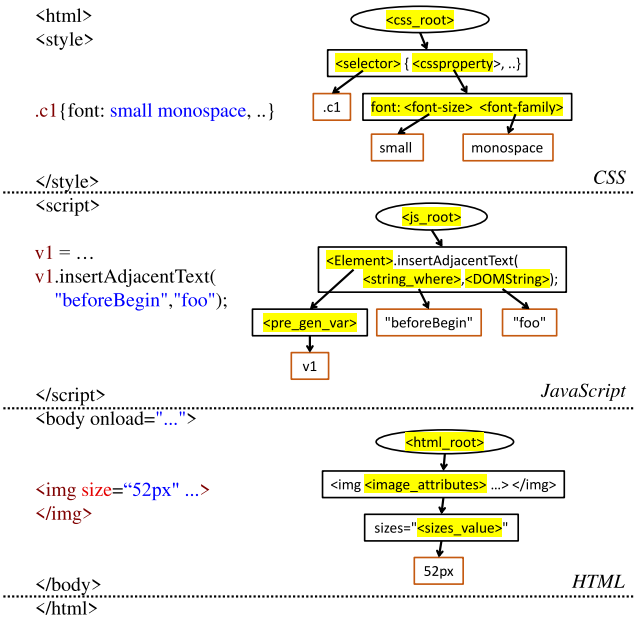


Fig. 2. Illustration of how grammar-based fuzzers generate an HTML document. The trees on the right side are derivation trees generated by the fuzzer. Symbols with a yellow background on nodes denote nonterminals. Edges denote production rules. The statements on the left side are instantiated from the derivation trees.

Fig. 2 takes Domato [Google 2017], the most successful browser fuzzer so far, as an example to illustrate grammar-based browser fuzzing. To generate a derivation tree, the fuzzer recursively expands production rules for each nonterminal until all nonterminals are replaced by terminals. For example, the *<cssproperty>* nonterminal is expanded by the production rule *<cssproperty>* → *font: <font-size> <font-family>*, and the *<font-size>* and *<font-family>* are recursively expanded